

Apigee Confidential BFF Architecture - Decision Framework

OAuth 2.1 Confidential BFF Pattern: Apigee as BFF vs. Apigee + Dedicated BFF Service

Background

This document evaluates two architectural approaches for implementing an OAuth 2.1-compliant Confidential BFF (Backend for Frontend) pattern using Apigee as the API Gateway and PingFederate as the Identity Provider.

Key question

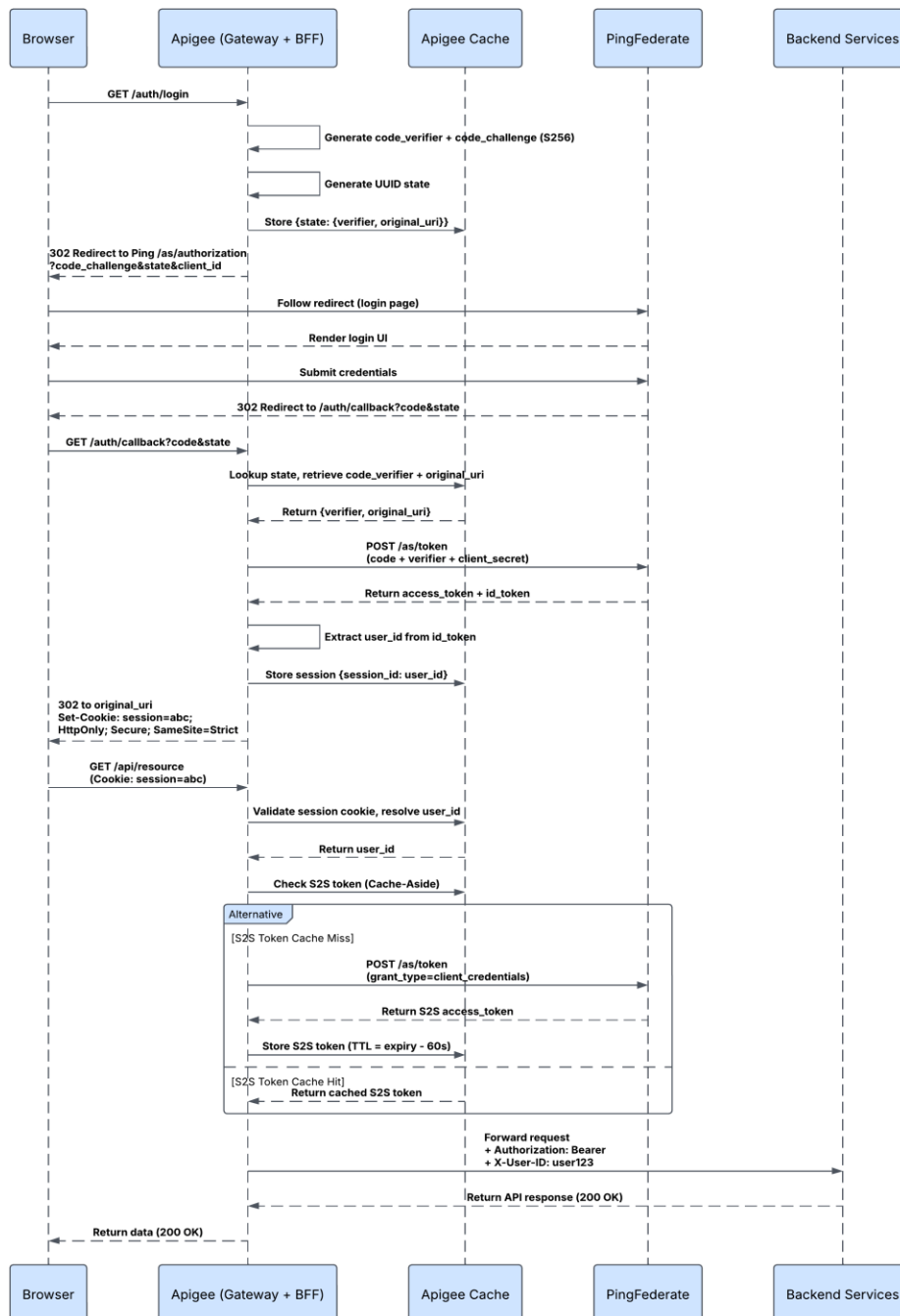
Should Apigee act as both the gateway AND the BFF (handling PKCE, token exchange, and session management), or should a dedicated BFF service sit behind Apigee and own all auth logic?

Category	Option A Apigee as BFF	Option B Apigee + Separate BFF	Winner
ARCHITECTURE			
Complexity	Single layer, fewer moving parts	Two services to deploy, configure, and operate	Option A
Separation of concerns	Gateway and auth logic mixed in one layer	Gateway stays a gateway. BFF owns all auth	Option B
Network hops	No extra hop. Apigee handles end to end	Extra hop: Apigee → BFF on every request	Option A
DEVELOPMENT			

Testability	Apigee JS policies are hard to unit test locally	BFF is a real service, so full test coverage possible	Option B
Developer experience	Auth logic spread across XML policies and JS snippets	Auth logic in one codebase, familiar stack (Node/Java)	Option B
Iteration speed	Apigee deployments are slower to push and test	BFF deploys independently, faster CI/CD cycle	Option B
OPERATIONS			
Session store	Apigee Cache -- limited TTL control, no pub/sub	Redis -- battle-tested, supports rotation and revocation	Option B
Observability	Auth traces buried inside Apigee analytics	BFF emits its own spans, logs, and metrics independently	Option B
Scaling	Both scale horizontally. Apigee is fully managed	Both scale horizontally. BFF runs on Cloud Run	Tie
SECURITY			
Token handling	Tokens live in Apigee Cache -- shared with gateway infra	Tokens isolated in BFF + Redis - clear security boundary	Option B
Secret management	client_secret stored in Apigee KVM -- less flexible rotation	BFF uses Secret Manager / env injection -- standard pattern	Option B
Auth extensibility	MFA step-up, device fingerprinting, risk scoring are painful	Any auth feature is a code change in the BFF	Option B
FIT			
Best for	Small teams, quick POC, low auth complexity, limited ops bandwidth	Enterprise scale, long-lived product, complex auth requirements	—

When to use	Only when Apigee is the identity server (no external IdP like Ping)	When an external IdP (Ping, Okta, Auth0) is in the architecture	—
-------------	---	---	---

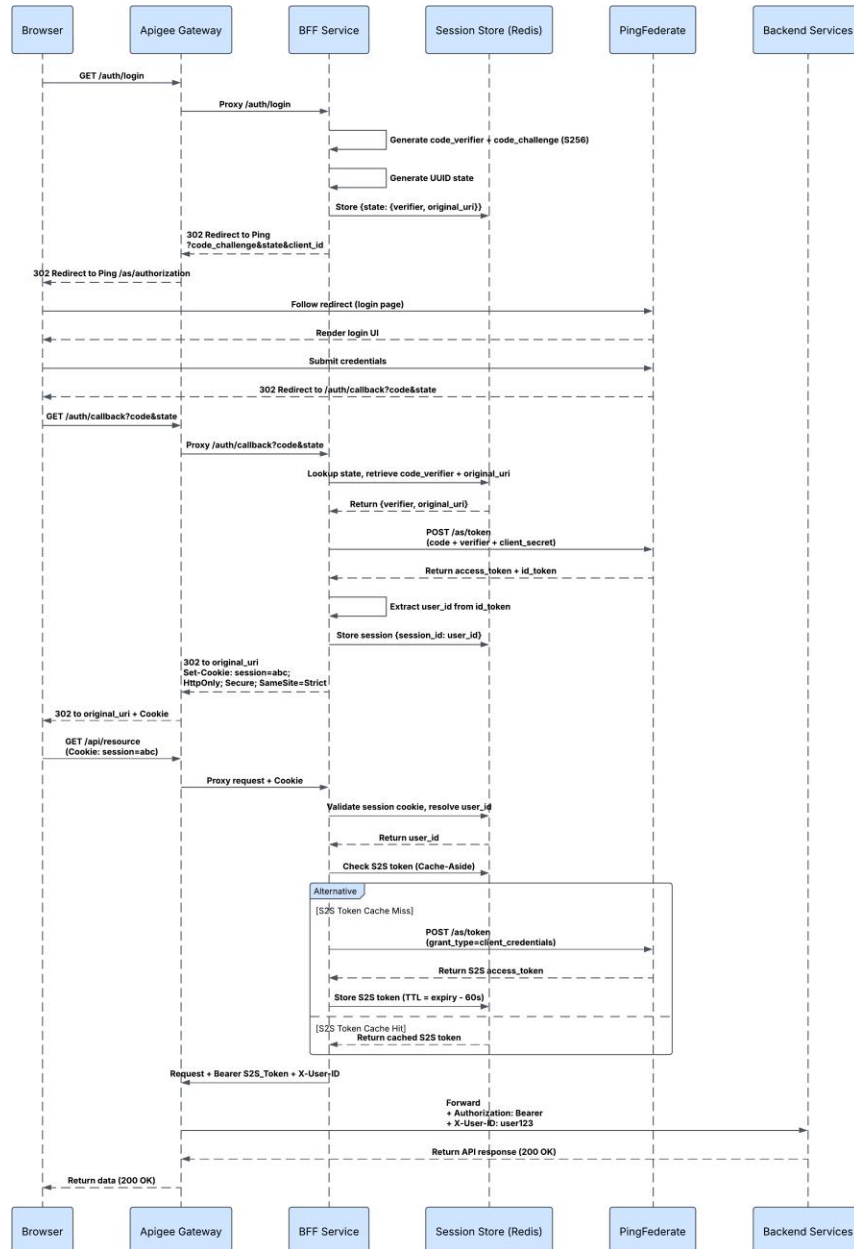
Sequence Diagram – Apigee as BFF



Full Diagram - [Apigee as BFF.png](#)

Above sequence diagram shows that Apigee acts as BFF as proposed, where Apigee is the orchestrator and manages sessions and tokens, along with acting as a cache layer ([UI User Request Flow.docx](#))

Sequence Diagram – Apigee as gateway and separate BFF



Full Diagram - [Apigee + BFF.png](#)

Above sequence diagram shows that Apigee acts as BFF as proposed, where Apigee is the orchestrator and manages sessions and tokens, along with acting as a cache layer ([UI User Request Flow.docx](#))

Recommendation

Option B: Apigee + Separate BFF Service

Apigee should own what a gateway owns: routing, rate limiting, TLS termination, and S2S token enforcement on outbound calls to backend services. The BFF service should own the full OAuth orchestration, PKCE generation, state management, token exchange with PingFederate, session management (Redis), and HTTP Only cookie issuance.